Журнал событий в PostgreSQL

Клиент передаёт целое число n от 0 до N. При успехе сервер отвечает n + 1. Если n уже поступало или n + 1 уже успешно обработано, возвращается ошибка и создаётся запись в журнале. Проверка повтора выполняется первой.

Один журнал вместо двух хранилищ

Каждое завершённое обращение записывается в events. Успешные числа определяются строками с outcome=OK. Проверки выполняются в SERIALIZABLE: PostgreSQL обнаруживает несовместимые параллельные операции. При конфликте приложение откатывает попытку и повторяет все проверки с новой транзакцией.

Данные

Объект	Назначение
events	Единственная таблица исходов запросов
request_history	Представление всех событий
processed_numbers	Представление событий с outcome=OK

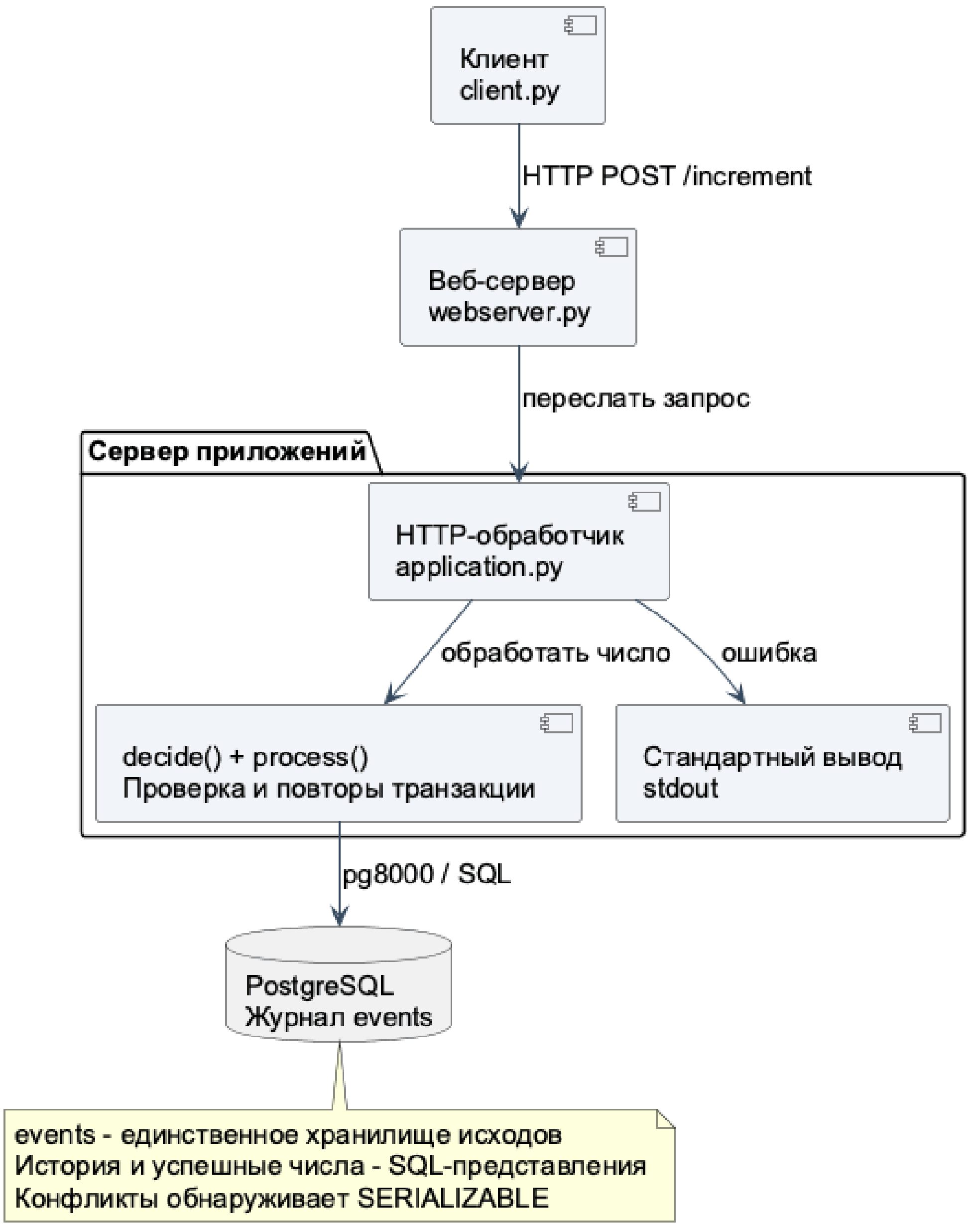
Ключевой фрагмент программы

```
cur.execute('SET TRANSACTION ISOLATION LEVEL SERIALIZABLE')
outcome = decide(cur, number)
cur.execute(
    'INSERT INTO events(request_id,number,outcome) VALUES(%s,%s,%s)',
    (request_id, number, outcome)
)
conn.commit()
```

Это фрагмент application.py. Полная программа также проверяет ввод, обрабатывает ошибки и отменяет незавершённую транзакцию. В базе сохраняется вход n, а не вычисленный ответ n + 1.

Диаграмма компонентов

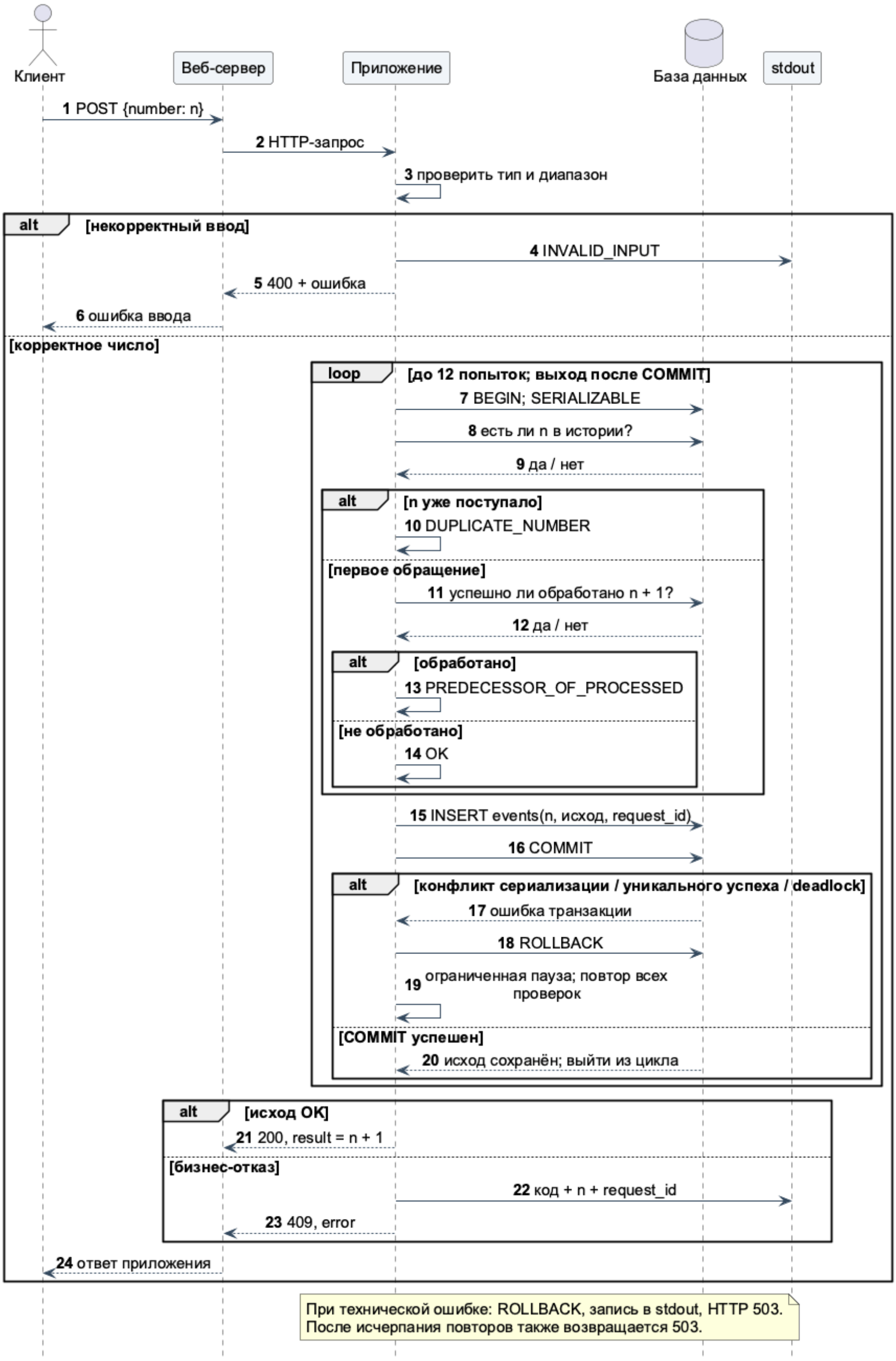
Клиент, веб-сервер и приложение выполняют разные роли. Веб-сервер только пересылает запрос. Приложение выбирает исход, база хранит историю. Представление, прикладная логика и данные образуют три логических уровня.



Веб-сервер и приложение - разные процессы, PostgreSQL - отдельный сервер. decide() определяет исход, process() отвечает за транзакцию и ограниченные повторы.

Диаграмма последовательностей

Время идёт сверху вниз. Ветви alt показывают разные результаты запроса. Цикл повторяет всю транзакцию после подтверждённого конфликта.



Корректное число с бизнес-отказом попадает в историю: повтор такого числа тоже считается повтором. Неверный JSON, тип или диапазон даёт ошибку ввода и не добавляет число в историю.

Примеры, проверка и запуск

Запросы по порядку	Ответ
5	200, результат 6
5 ещё раз	409, DUPLICATE_NUMBER
4	409, PREDECESSOR_OF_PROCESSED
6	200, результат 7

Все 11 интеграционных тестов прошли. В тестах использовались два процесса приложения и два веб-сервера с настоящей PostgreSQL. Из 20 одновременных одинаковых запросов ровно один завершился успешно, остальные получили ошибку повтора.

Проверены 15 некорректных входов, границы диапазона, 10 пар соседних чисел, сохранение истории после перезапуска, откат записи и недоступность сервисов. Обе бизнес-ошибки проверены в stdout и отсутствуют в stderr. run_local.py проверен отдельно: stdout приложения виден через запускатель.

Дополнительный тест намеренно замедляет запись в базе, чтобы создать конфликт. Проверено, что программа повторяет транзакцию и сохраняет ровно два исхода для двух запросов. Повторы ограничены 12 попытками; после исчерпания возвращается 503.

Запуск из каталога варианта

```
python3 -m pip install -r requirements.txt
python3 run_local.py --pg-bin /path/to/postgresql/bin
# In another terminal:
python3 client.py 5
# Integration tests:
python3 test_integration.py --pg-bin /path/to/postgresql/bin
```

Python 3.12+. Нужны pg8000 и установленный PostgreSQL; проверено с PostgreSQL 17.6. Данные сохраняются в .runtime; Ctrl-C останавливает процессы. Порты и каталог можно изменить аргументами. Это локальный учебный сервис без авторизации и TLS.

Результаты: test_results.json и launch_results.json. Полные команды, формат HTTP и имена файлов: README.txt. Диаграммы доступны также в редактируемом .puml и в SVG.

Условие: Achievement #2, слайд 17 второй лекции. Код, схема БД и формат HTTP разработаны для варианта. Технический источник: PostgreSQL: SERIALIZABLE.